

Exercise #2: Benchmarking RDMA Throughput over InfiniBand

Authors: Yitzchak Kupinsky 212155444 & Shimon Fishman 209086446

1. Measurement Methodology & Implementation

Experimental Setup & Key Exchange Before we could start benchmarking the RDMA throughput, we first had to establish the connection between our machines. We ran our tests on the department's `mlx-stud` servers, which are equipped with InfiniBand network cards. Because InfiniBand Queue Pairs (QPs) cannot discover each other on their own, we implemented an out-of-band setup phase. We opened a standard TCP socket between the client and server to exchange our hardware parameters, for example exchange the virtual memory addresses (VADDR), and the remote access keys (RKEYs). Once both sides successfully traded and parsed these keys to transition their QPs into the RTS (Ready To Send) state, we shut down the TCP socket and let the InfiniBand hardware take over entirely.

Warm-up Cycles To ensure our timing calculations were accurate and not skewed by hardware wake-up times, we implemented a strict warm-up phase in our code. Before starting the high-resolution timer for any new message size, the client fires off 100 warm-up cycles. Basically, we use these warm-up cycles to 'wake up' the hardware before the real test begins. It gets the memory addresses cached, opens up the data pipes on the motherboard, and gets the RAM ready to go. That way, the exact millisecond we start our stopwatch, the system is already sprinting at top speed.

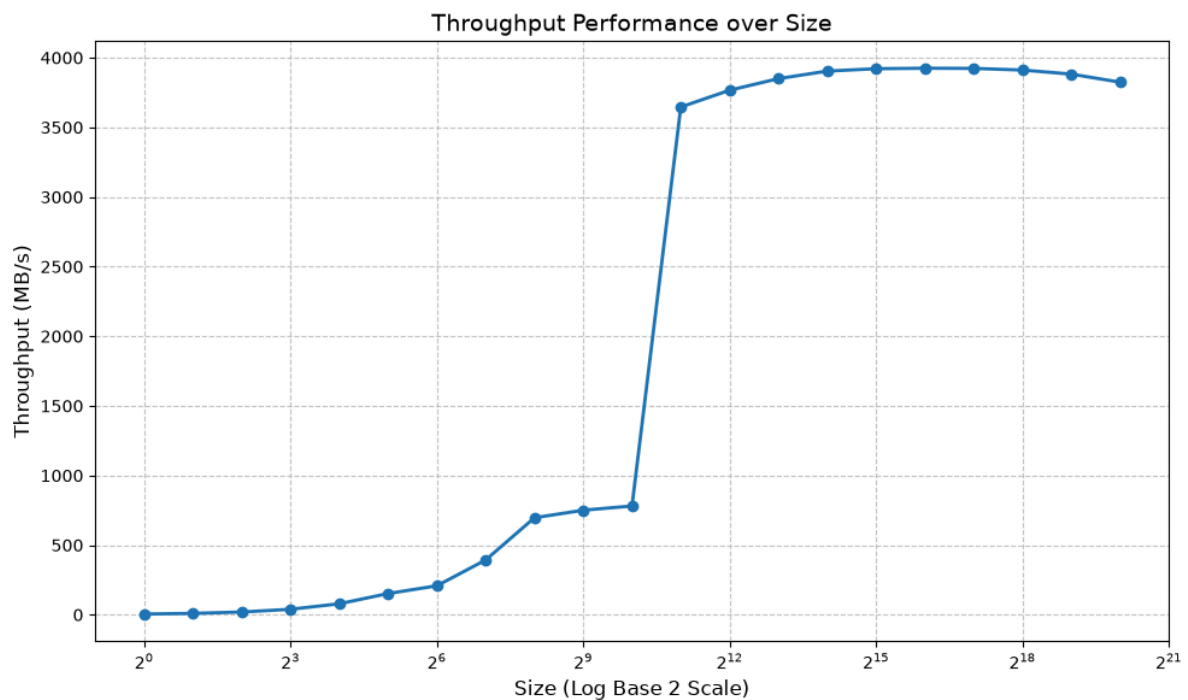
Dynamic Batch Sizing Instead of hardcoding a static number of messages for our `for` loop, we wrote the client to dynamically calculate the batch size so that exactly 64 MB of total data is transmitted during every single test case.

- **For small messages:** When we tested tiny payloads, random network spikes can really mess with the timing. We also had to account for the "inline" factor: our network cards pack small messages directly into the send queue instead of fetching them from RAM. But as our loop increased the message sizes, they eventually crossed the inline limit, forcing the hardware to change how it behaves. We believe this shift can introduce even more jitter and noise into the benchmark. To fix this, we sent a massive quantity of these smaller messages, which helps averaging out the noise and gives us a more stable baseline.
- **For large messages:** On the flip side, for giant 1MB payloads, we dropped the batch size. Sending millions of 1MB messages would just take forever to finish. Plus, because the actual data transfer takes so much longer at that size, that tiny bit of background network noise (and the inline issue, which doesn't apply to big messages anyway) is less significant. Sending fewer messages gave us a perfectly accurate read on our max bandwidth without making us sit around all day.

2. Performance Results

Raw Output Data:

```
yit.kup@mlx-stud-04:~/WorkshopCommunicationNetworks_EX2 $ ./client -p 4564 mlx-stud-03
local address: LID 0x0003, QPN 0x00013c, PSN 0xc92dc7, GID ::, VADDR 00007f5518bc4010, RKEY 0017cbbc
inline data support enabled (max_inline_data = 60)
remote address: LID 0x0001, QPN 0x000119, PSN 0xc2165c, GID ::, VADDR 00007f2b21213010, RKEY 0017cbbc
1      4.89   MB/s
2      9.74   MB/s
4     19.54   MB/s
8     39.03   MB/s
16    78.62   MB/s
32   152.02   MB/s
64   207.76   MB/s
128  393.08   MB/s
256  696.33   MB/s
512  750.57   MB/s
1024 780.81   MB/s
2048 3645.93   MB/s
4096 3766.39   MB/s
8192 3849.72   MB/s
16384 3903.82   MB/s
32768 3920.90   MB/s
65536 3924.61   MB/s
131072 3923.22   MB/s
262144 3911.37   MB/s
524288 3882.45   MB/s
1048576 3824.43   MB/s
Client Done.
yit.kup@mlx-stud-04:~/WorkshopCommunicationNetworks_EX2 $ |
```



3. Analysis

When we plotted the results, the jump from our TCP benchmark in Exercise 1 was significant. We actually got to see the benefits of bypassing the OS kernel in real-time. Looking at the graph, our hardware basically went through four distinct phases:

- **Hitting the Message Limit (1B to 1KB):** At the start, with tiny messages, the total bandwidth was low but doubled perfectly at each step (4.89 -> 9.74 -> 19.54 MB/s). We realized the bottleneck here was the actual number of messages the hardware could handle per second (the IOPS limit). Since the system was maxed out at processing a certain amount of messages a second, doubling the size of each message just naturally doubled our total throughput.
- **The Big Jump (1KB to 2KB):** The craziest part of the graph is the massive 4.6x spike in speed between 1KB and 2KB. This happened because the size of our actual data grew to a point where the network headers just stopped being a major bottleneck. Even though we didn't measure the exact header sizes, it's clear that at this threshold, the fixed overhead of wrapping packets became way less significant compared to the huge chunks of data being transferred. This allowed the network card to efficiently pull memory straight over the motherboard without being dragged down by overhead.
- **Crushing the TCP Limit:** This is where RDMA really shined. In Exercise 1, our TCP test maxed out at 1278.61 MB/s because the CPU was wasting time copying memory and switching contexts in the operating system. By using one-sided `RDMA_WRITE` operations, we bypassed the server's CPU completely. We basically maxed out the physical wire, peaking at 3924.61 MB/s for 64KB messages. That's more than triple the speed (a 307% increase) on the exact same physical hardware.